

Python
for DA

Polars: работа с несколькими таблицами



Преподаватель

Портрет

Имя Фамилия

Текущая должность

Количество лет опыта

Какой у Вас опыт - ключевые кейсы

Самые яркие проекты

Дополнительная информация по вашему усмотрению

Корпоративный e-mail

Социальные сети (по желанию)

Важно

- 

Камера должна быть включена на протяжении всего занятия
- 

В течение занятия вопросы задавать в чате или когда преподаватель спрашивает, есть ли у Вас вопросы
- 

Вести себя уважительно и этично по отношению к остальным участникам занятия
- 

Организационные вопросы по обучению решаются с кураторами, а не на тематических занятиях
- 

Во время занятия будут интерактивные задания, будьте готовы включить камеру или демонстрацию экрана по просьбе преподавателя

Повторение

-  Обработка пропусков
-  Изменение типов данных. Кастинг
-  Операции агрегации
-  Повторение describe для одного поля
-  Groupby
-  Оконные функции
-  Уникальные значения
-  Сложная фильтрация
-  Условия

План занятия

- Пример
- Join
- Asof Join
- Merge_sorted
- Конкатенация
- Unpivot



ОСНОВНОЙ БЛОК





Пример



Пример



Создадим примеры датасетов для демонстрации операций соединения

Пример

```
import polars as pl
import numpy as np

# Create a random seed for reproducibility
np.random.seed(42)

# Dataset 1: User information
data1 = {
    'user_id': np.arange(1, 101),
    'name': np.random.choice(['Alice', 'Bob', 'Charlie', 'David'], 100)
}
df1 = pl.DataFrame(data1)

# Dataset 2: User age information
data2 = {
    'user_id': np.arange(51, 151),
    'age': np.random.randint(18, 65, size=100)
}
df2 = pl.DataFrame(data2)

# Dataset 3: User scores
data3 = {
    'user_id': np.random.randint(1, 101, size=100),
    'score': np.random.randint(200, 1000, size=100)
}
df3 = pl.DataFrame(data3)
```



ВОПРОСЫ





Join



Типы объединений, доступные в Polars

Inner Join

Left Join

Outer Join

Cross Join

Semi Join

Anti Join

Inner Join



Описание

Возвращает строки, которые имеют совпадающие значения в обеих таблицах. Используется, когда нужно получить только те записи, которые имеют соответствующие записи в обоих объединяемых DataFrames.

Пример

```
inner_joined_df = df1.join(df2, on='user_id', how='inner')
print("Inner Join Result:\n", inner_joined_df.head())
```

Left Join



Описание

Возвращает все строки из левой таблицы и совпадающие строки из правой таблицы. Если совпадений нет, результатом будет `NULL` на стороне правой таблицы.

Пример

```
left_joined_df = df1.join(df2, on='user_id', how='left')
print("Left Join Result:\n",
      left_joined_df.sample(10).head())
```

Outer Join



Описание

Возвращает все строки из левой и правой таблиц. Если совпадений нет, результатом будет `NULL` на стороне таблицы без совпадений. Заполняет недостающие значения в столбцах соединения.

Пример

```
full_outer_joined_df = df1.join(df2, on='user_id',
                                how='outer')
print("Full Outer Join Result:\n",
      full_outer_joined_df.sample(10).head())
```

Cross Join



Описание

Получает декартово произведение двух таблиц. Каждая строка первой таблицы объединяется со всеми строками второй таблицы

Пример

```
df_brands = pl.DataFrame({
    'brand': ['lacoste', 'prada', 'calvin klein']
})
df_sizes = pl.DataFrame({
    'sizes': ['S', 'M', 'L']
})
cross_joined_df = df_brands.join(df_sizes, how='cross')
print("Cross Join Result (first 5 rows):\n",
      cross_joined_df)
```

Semi Join



Описание

Возвращает все строки из левой таблицы, для которых есть совпадающие ключи в правой таблице, но не включает ни одного столбца из правой таблицы.

Пример

```
semi_joined_df = df1.join(df2, on='user_id', how='semi')
print(f"Semi Join Result {semi_joined_df.shape}:\n",
      semi_joined_df.head())
```

Anti Join



Описание

Возвращает все строки из левой таблицы, для которых нет совпадающих ключей в правой таблице.

Пример

```
anti_joined_df = df1.join(df2, on='user_id', how='anti')
print(f"Anti Join Result {anti_joined_df.shape}:\n",
      anti_joined_df.head())
```



ВОПРОСЫ





Asof Join





Соединение asof в Polars

Это специализированный тип операции соединения, предназначенный в первую очередь для данных временных рядов, где целью является объединение двух наборов данных на основе ближайшего ключа, а не точного совпадения.

Основные характеристики Asof Join



Ближайшее совпадение ключей



Направленность



Требование сортировки



Обработка несовпадающих ключей



backward



forward



nearest



Пример с использованием Polars

```
import polars as pl

df_q = pl.DataFrame({
    "time": ["2024-01-01 10:00", "2024-01-01 10:02", "2024-01-01 10:05"],
    "quantity": [10, 20, 30]
}).with_columns(pl.col("time").str.strptime(pl.Datetime)).sort('time')

df_p = pl.DataFrame({
    "time": ["2024-01-01 10:01", "2024-01-01 10:03", "2024-01-01 10:06"],
    "price": [100, 200, 300]
}).with_columns(pl.col("time").str.strptime(pl.Datetime)).sort('time')

# Выполнение объединения по принципу asof
result = df_q.join_asof(df_p, on="time")
print(result)
```



tolerance

Это важная функция, которая позволяет указать предел расстояния между ключами объединения, чтобы строки считались совпадающими.

Параметр допуска



В Polars может быть задан с помощью объекта `timedelta` при работе с типами данных, похожими на время. Это позволяет задавать допуски в единицах времени.

Пример использования tolerance

```
from datetime import timedelta

# Выполняем объединение asof с допуском 1 минута
result = df_q .join_asof(df_p, on=«time», tolerance=timedelta(minutes=1))
print(result)
```



ВОПРОСЫ





Merge_sorted



Merge_sorted

Это специализированный метод, предназначенный для объединения двух уже отсортированных DataFrame на основе указанного ключа, гарантируя, что результирующий DataFrame останется отсортированным.

Ключевые особенности merge_sorted



Сохранение порядка

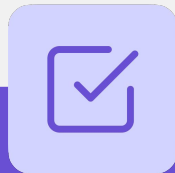


Требование к предварительно отсортированным рамкам данных



Совпадение схем

Основной синтаксис функции merge_sorted



```
DataFrame.merge_sorted(other, key)
```

- other: Другой DataFrame, который должен быть объединен с текущим DataFrame
- ключ: Имя столбца или список имен столбцов, по которым отсортированы DataFrame и которые будут использоваться в качестве ключа слияния

Пример использования merge_sorted

```
df11 = pl.DataFrame({
    "Name": ["John", "Joseph", "Albert"],
    "Age": [18, 15, 29]
}).sort("Age")

df12 = pl.DataFrame({
    "Name": ["Ema", "Andrew", "Michel"],
    "Age": [22, 30, 16]
}).sort("Age")

df13 = df11.merge_sorted(df12, "Age")
print(df13)
```



ВОПРОСЫ





Конкатенация



Конкатенация

Это операция, которая используется для объединения DataFrames или Series как по вертикали (укладывая их друг на друга), так и по горизонтали (растягивая их по сторонам).



Вертикальная конкатенация

Это операция, которая объединяет фреймы DataFrames, складывая их друг на друга.

Вертикальная конкатенация

```
df21 = pl.DataFrame({
    "Name": ["Alice", "Bob"],
    "Age": [25, 30]
})

df22 = pl.DataFrame({
    "Name": ["Charlie", "David"],
    "Age": [35, 40]
})

df_vertical = pl.concat([df21, df22])
print(df_vertical)
```

Пример

shape: (4, 2)

Name	Age
---	---
str	i64
Alice	25
Bob	30
Charlie	35
David	40

Вывод



Горизонтальная конкатенация

Это операция, которая объединяет фреймы
DataFrames, растягивая их рядом друг с другом.

Горизонтальная конкатенация

```
df23 = pl.DataFrame({
    "City": ["New York", "Los Angeles"],
    "Occupation": ["Engineer", "Doctor"]
})

df_horizontal = pl.concat([df21, df23],
    how="horizontal")
print(df_horizontal)
```

Пример

shape: (2, 4)

Name	Age	City	Occupation
---	---	---	---
str	i64	str	str
Alice	25	New York	Engineer
Bob	30	Los Angeles	Doctor

Вывод



Диагональная конкатенация

Это мощный метод объединения DataFrames, которые могут иметь разные схемы. Эта операция не только добавляет записи из нескольких DataFrames, но и управляет различными столбцами, вводя нулевые значения там, где данные отсутствуют в исходных DataFrames.

Диагональная конкатенация

```
df_diagonal = pl.concat(
    [
        df21,
        df23,
    ],
    how="diagonal",
)
print(df_diagonal)
```

shape: (4, 4)

Name	Age	City	Occupation
---	---	---	---
str	i64	str	str
Alice	25	null	null
Bob	30	null	null
null	null	New York	Engineer
null	null	Los Angeles	Doctor

Пример

Вывод

Конкатенация

Имена столбцов

Типы данных

Для вертикальной конкатенации имена столбцов в DataFrames должны совпадать. Если они не совпадают, возможно, потребуется переименовать столбцы перед конкатенацией.

Конкатенация

Имена столбцов

Типы данных

Типы данных конкатенируемых столбцов должны быть совместимы. При несовпадении Polars может выполнить неявное приравнивание типов, что может привести к потере данных или ошибкам.

Rechunking



Если вы не хотите, чтобы Polars повторно копировал конкатенированный DataFrame, укажите `rechunk = False` при выполнении конкатенации



ВОПРОСЫ





Unpivot





Unpivot

Это функция, используемая для преобразования DataFrame из широкого формата в длинный. Разворачивает DataFrame, превращая столбцы в строки.

Переменные-идентификаторы (index)

Это столбцы в DataFrame, которые используются для идентификации каждой строки.



Важно:

Переменные-идентификаторы остаются в вертикальном формате и при необходимости дублируются в результирующем длинном формате DataFrame.

Переменные значения (on)

Эти столбцы не имеют поворотов и преобразуются в два столбца в длинном формате - один для имен переменных и один для значений.



Важно:

Если on не указан, все столбцы, не заданные как index, используются как переменные значения.

Имя переменной и имя значения

Можно задать имена для новых столбцов, в которых будут храниться имена переменных и их значения.



Важно:

Если они не указаны, используются значения по умолчанию («variable» для имен неповоротных столбцов и «value» для соответствующих значений).

Пример использования Unpivot

```
df = pl.DataFrame({
    "Year": [2020, 2021],
    "Product A": [100, 150],
    "Product B": [90, 120]
})
print(df)
melted_df = df.unpivot(index="Year",
on=["Product A", "Product B"],
variable_name="Product",
value_name="Sales")
print(melted_df)
```

shape: (4, 3)

Year	Product	Sales
---	---	---
i64	str	i64
2020	Product A	100
2021	Product A	150
2020	Product B	90
2021	Product B	120

Пример

Вывод

Unpivot

Целостность данных

Производительность

Обработка пропущенных
данных

Переменные-идентификаторы (index) однозначно идентифицируют каждую строку в исходных данных, чтобы избежать проблем с дублированием данных в новом DataFrame.

Unpivot

Целостность данных

Производительность

Обработка пропущенных
данных

Для повышения
производительности
рекомендуется перед unpivot
отфильтровать или
уменьшить размер
DataFrame, если это
возможно.

Unpivot

Целостность данных

Производительность

Обработка пропущенных
данных

В зависимости от ваших
данных и контекста
операции, вам может
понадобиться заполнить или
обработать нули после
преобразования.



ВОПРОСЫ





ПРАКТИЧЕСКАЯ РАБОТА



Задание

1. Почему `merge_sorted` удобен для взаимодействия с временными рядами?
2. Что делает этот код и сколько строк будет в результирующей таблице?

```
import polars as pl

df_employees = pl.DataFrame({
    "employee_id": [1, 2, 3, 4],
    "name": ["Alice", "Bob", "Charlie", "David"],
    "department_id": [101, 102, 103, 101]
})

df_departments = pl.DataFrame({
    "department_id": [101, 102, 104],
    "department_name": ["HR", "IT", "Finance"]
})

result = df_employees.join(df_departments, on="department_id", how="left")
```

Задание

3. Где ошибка в коде?

```
import polars as pl

df_students = pl.DataFrame({
    "student_id": [1, 2, 3],
    "student_name": ["Alice", "Bob", "Charlie"]
})

df_grades = pl.DataFrame({
    "id": [1, 2, 4],
    "grade": ["A", "B", "C"]
})

result = df_students.join(df_grades, on="student_id", how="inner")
```



ВОПРОСЫ



Заключение

